

UNIVERSITÉ PARIS CITÉ

Master 1 Machine Learning pour la Science des Données, Année 2025/2026

Projet Tuteuré

LLM-Enabled Few-Shot Clustering

Groupe : Wissal Zghrata Rim Zeidan Waël Akil

Article étudié :

Vijay Viswanathan, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu & Graham Neubig (2024).

Large Language Models Enable Few-Shot Clustering.

Transactions of the Association for Computational Linguistics, 12, 321–333.

doi:10.1162/tacl_a_00648 | arXiv:2307.00524

Code source : <https://github.com/Dr2xU/few-shot-clustering>

Table des matières

Introduction	1
1 État de l'art	1
1.1 Le clustering de texte : des méthodes classiques aux représentations neuronales	1
1.1.1 Approches classiques : représentations creuses et modèles probabilistes	1
1.1.2 Clustering sur embeddings neuronaux pré-entraînés	1
1.1.3 Clustering profond et apprentissage contrastif	2
1.2 Le clustering semi-supervisé : contraindre pour satisfaire l'utilisateur	2
1.2.1 Motivation et typologie des interactions expert-algorithme	2
1.2.2 Pairwise constraint clustering : PCKMeans et Explore-Consolidate	2
1.2.3 La limite commune : le coût de l'annotation humaine	3
1.3 Les LLMs comme oracles, simulateurs et guides du clustering	3
1.3.1 Les LLMs comme simulateurs et évaluateurs : un cadre conceptuel	3
1.3.2 Panorama des LLMs appliqués au clustering de texte	3
1.3.3 Positionnement de Viswanathan et al. (2024) dans ce paysage	4
1.4 Synthèse et positionnement de l'état de l'art	5
2 Description de l'approche	6
2.1 Tâches et datasets	6
2.1.1 Canonicalisation d'entités	6
2.1.2 Clustering d'intentions et de tweets	6
2.2 Les trois méthodes proposées	7
2.2.1 Avant le clustering : enrichissement par keyphrases	7
2.2.2 Pendant le clustering : PCKMeans avec pseudo-oracle LLM	7
2.2.3 Après le clustering : correction des assignations à faible confiance	8
2.3 Baselines et protocole expérimental	8
2.4 Résultats rapportés par les auteurs	9
3 Analyse critique	9
3.1 Points forts de l'approche	10
3.1.1 Un pipeline modulaire	10
3.1.2 Enrichissement sémantique	10
3.1.3 Supervision via pseudo-oracle	10
3.1.4 Correction ciblée des assignations	10
3.2 Limitations identifiées	11
3.2.1 Des choix non validés et un coût éludé	11
3.2.2 Fragilité du pseudo-oracle face au bruit de décision	11
3.2.3 Biais dans la sélection des baselines	12
3.2.4 Fragilités structurelles de la correction ciblée	12
3.3 Analyse des évaluations proposées par les auteurs	13
3.3.1 Ablations et décomposition du signal utile	13
3.3.2 Coût et scalabilité de l'argument économique	13
3.3.3 Adéquation des métriques au problème étudié	14
3.4 Remarques sur les prompts	14
4 Évaluations complémentaires	15
4.1 Ablation encodeurs : le gain vient-il du LLM ou de la représentation ?	15
4.1.1 Motivation	15
4.1.2 Protocole	15
4.1.3 Résultats	16
4.1.4 Analyse	16
4.1.5 Saturation du budget de contraintes	16
4.2 Cohérence logique des contraintes : un signal monotone	17

4.2.1	Motivation	17
4.2.2	Protocole	17
4.2.3	Résultats	17
4.2.4	Analyse	17
4.3	Baseline géométrique : le LLM est-il nécessaire pour la post-correction ?	17
4.3.1	Motivation	17
4.3.2	Protocole	18
4.3.3	Résultats	18
4.3.4	Analyse	18
4.4	Généralisation hors distribution : 20 Newsgroups	19
4.4.1	Motivation	19
4.4.2	Résultats préliminaires	19
4.4.3	Analyse	19
4.5	Synthèse des évaluations complémentaires	19
	Conclusion	20

Introduction

Ce rapport présente l’analyse de l’article **Large Language Models Enable Few-Shot Clustering** de Viswanathan et al. (2024), publié dans Transactions of the Association for Computational Linguistics. Les auteurs y explorent comment un grand modèle de langage peut être utilisé comme pseudo-oracle pour guider le clustering de texte semi-supervisé en intervenant à trois stades du pipeline : enrichissement des représentations en amont, génération de contraintes pendant le clustering et correction des assignations en aval. Leur approche se distingue par sa simplicité méthodologique et par l’argument économique qu’elle avance en faveur d’un oracle automatique plutôt qu’humain.

Notre travail se structure en quatre temps. La section 1 dresse un état de l’art du clustering de texte, du clustering semi-supervisé et des approches récentes mobilisant des LLMs. La section 2 décrit en détail les trois méthodes proposées par les auteurs ainsi que les tâches et datasets sur lesquels elles sont évaluées. La section 3 en propose une analyse critique, identifiant les points forts, les limitations et les biais potentiels des évaluations rapportées. La section 4 présente nos résultats de reproduction et nos évaluations complémentaires, conçues pour tester la robustesse de l’approche sur des axes que les auteurs n’ont pas couverts.

1 État de l’art

Le clustering de texte consiste à regrouper un ensemble de documents en sous-ensembles homogènes sans annotation préalable. Des **représentations creuses classiques** (TF-IDF) aux **embeddings neuronaux denses**, les progrès en qualité de représentation n’ont pas résolu une difficulté fondamentale : sans connaissance de l’intention de l’utilisateur plusieurs partitionnements sont également valides pour un même corpus et l’algorithme ne peut pas savoir lequel correspond à l’attente de l’expert. Le **clustering semi-supervisé** répond à cette sous-spécification en permettant à l’utilisateur d’injecter des contraintes, typiquement des paires de documents qui doivent ou ne doivent pas appartenir au même cluster mais la qualité du résultat dépend alors du volume d’annotations que l’expert est prêt à fournir. L’arrivée des **grands modèles de langage instruction-tunés** a ouvert une troisième voie : interroger un LLM comme pseudo-oracle pour produire à faible coût les contraintes qu’un annotateur fournirait à grand coût. Cet état de l’art retrace ces trois familles et situe l’article étudié au sein de cette littérature.

1.1 Le clustering de texte : des méthodes classiques aux représentations neuronales

1.1.1 Approches classiques : représentations creuses et modèles probabilistes

La formulation standard du clustering de texte repose sur des vecteurs creux de type TF-IDF pour encoder les documents et des algorithmes de partitionnement comme k-means pour les regrouper (Aggarwal et Zhai, 2012). La représentation TF-IDF associe à chaque document un vecteur dont chaque composante mesure l’importance d’un terme t du vocabulaire :

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \cdot \log \frac{|D|}{|\{d' \in D : t \in d'\}|}$$

L’algorithme k-means (Lloyd, 1982) partitionne ces vecteurs en K clusters en minimisant la distorsion intra-cluster :

$$\arg \min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

La variante k-means++ (Arthur et Vassilvitskii, 2007) améliore l’initialisation en sélectionnant des centroïdes éloignés les uns des autres, ce qui stabilise les résultats. C’est cette variante qui reste utilisée comme baseline dans Viswanathan et al. (2024). Les modèles probabilistes comme LDA (Blei et al., 2003) offrent une alternative en représentant chaque document comme un mélange de sujets latents, mais posent des hypothèses fortes sur la structure générative des textes, notamment pour les textes courts.

Ces méthodes souffrent de deux limitations structurelles. Les représentations creuses ne capturent pas la sémantique : deux documents qui expriment la même idée avec des vocabulaires différents apparaissent éloignés dans l’espace TF-IDF. Par ailleurs, dans des espaces de grande dimension, les distances euclidiennes deviennent peu discriminantes, ce qui dégrade la qualité du partitionnement.

1.1.2 Clustering sur embeddings neuronaux pré-entraînés

L’arrivée des modèles de langage pré-entraînés a substitué aux vecteurs creux des représentations denses de faible dimension qui encodent la sémantique. BERT (Devlin et al., 2019) a posé les bases de cette transition. Sentence-BERT

(Reimers et Gurevych, 2019) a spécialisé cette idée pour les tâches de similarité via une architecture siamoise. DistilBERT (Sanh et al., 2019) en offre une version compressée, utilisée par Viswanathan et al. pour le dataset Tweet.

Instructor (Su et al., 2022) introduit une évolution importante : le conditionnement par instruction. Le modèle accepte en entrée une instruction textuelle décrivant la tâche visée et ajuste sa sortie en conséquence ce qui permet de produire des embeddings adaptés à différents usages sans réentraînement. Viswanathan et al. (2024, section 4.1) mobilisent Instructor pour les datasets Bank77 et CLINC avec l’instruction **"Represent utterances for intent classification"**. Depuis 2022, plusieurs encodeurs modernes ont étendu cette lignée : E5 (Wang et al., 2022) via un pré-entraînement contrastif faiblement supervisé, BGE (Xiao et al., 2023) avec une stratégie multi-étapes et mxbai-embed-large-v1 (Lee et al., 2024) qui atteint des performances compétitives avec des modèles propriétaires tout en conservant une empreinte modeste (334M de paramètres).

1.1.3 Clustering profond et apprentissage contrastif

Une troisième famille apprend conjointement la représentation et le partitionnement. DEC (Xie et al., 2015) utilise un autoencodeur dont un objectif auxiliaire raffine simultanément les embeddings et les assignations de cluster. SCCL (Zhang et al., 2021) y intègre de l’apprentissage contrastif, produisant des embeddings plus discriminants pour les textes courts. ClusterLLM (Zhang et al., 2023) dont le signal d’entraînement provient directement d’un LLM, est décrit en détail dans la section 1.3.2 comme concurrent principal de l’article étudié.

Ces approches produisent un encodeur spécialisé pour la tâche et le dataset considérés, au détriment d’un coût accru en complexité d’implémentation et en ressources de calcul. C’est précisément ce contraste que l’article exploite : leur méthode revendique de rester *plug-in* en s’appuyant sur un encodeur gelé et en injectant le signal LLM par d’autres canaux, évitant ainsi la phase de fine-tuning qui caractérise DEC, SCCL et ClusterLLM.

1.2 Le clustering semi-supervisé : contraindre pour satisfaire l’utilisateur

1.2.1 Motivation et typologie des interactions expert-algorithme

Le clustering non supervisé est un problème intrinsèquement sous-spécifié. Il produit un partitionnement parmi plusieurs possibles sans garantie que celui-ci corresponde à l’attente de l’utilisateur. Caruana (2013) résume cette contradiction par la formule *probably approximately useless*, une inversion du célèbre cadre *probably approximately correct* de l’apprentissage statistique. Sa thèse mérite d’être nuancée : ce qu’il pointe n’est pas l’inutilité de tout clustering mais sa sous-spécification. Deux experts confrontés au même corpus peuvent légitimement attendre deux regroupements distincts (par sujet ou encore par registre de langue) et l’algorithme n’a accès à l’intention d’aucun des deux. Le clustering semi-supervisé répond à cette ambiguïté en permettant à l’utilisateur d’injecter un volume modeste de contraintes partielles qui suffisent à désambiguïser son intention (Bae et al., 2020).

La littérature recense cinq formes d’interaction expert-algorithme :

- Le **seeding** (Basu et al., 2002) qui initialise les centroïdes avec des exemples prototypiques étiquetés. Cette approche est simple mais suppose que l’expert puisse produire des exemples suffisamment discriminants pour chaque cluster attendu.
- Les **contraintes de paires** *must-link* / *cannot-link* (Wagstaff et Cardie, 2000) qui spécifient des relations entre paires de points sans étiquetage individuel. L’expert n’a pas besoin de connaître a priori les clusters attendus, il lui suffit d’exprimer des jugements relatifs.
- Le **feedback sur les features** (Dasgupta et Ng, 2010) qui indique quelles dimensions sont pertinentes pour le partitionnement visé.
- Les opérations **split** / **merge** (Awasthi et al., 2013) qui corrigent le clustering après une première passe. L’expert peut fusionner deux clusters qu’il juge redondants ou scinder un cluster qu’il juge trop hétérogène.
- Le **lock** / **refine** (Coden et al., 2017) qui verrouille les clusters satisfaisants pour ne réorganiser que les autres. Cette stratégie est utile lorsqu’une partie du partitionnement converge plus vite que le reste.

Toutes ces familles supposent que l’expert est disposé à fournir un nombre suffisant d’interactions pour que l’algorithme converge. Cette hypothèse est précisément celle que la section 1.2.3 remettra en question.

1.2.2 Pairwise constraint clustering : PCKMeans et Explore-Consolidate

Le *pairwise constraint clustering* s’est imposé comme le paradigme dominant du clustering semi-supervisé. L’algorithme PCKMeans (Basu et al., 2004) étend la fonction objectif de k-means en ajoutant deux termes de pénalité qui sanctionnent les violations de contraintes :

$$J_{\text{PCKMeans}} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 + \sum_{(x_i, x_j) \in \mathcal{M}} w_{ij} \cdot \mathbb{1}[l_i \neq l_j] + \sum_{(x_i, x_j) \in \mathcal{C}} \bar{w}_{ij} \cdot \mathbb{1}[l_i = l_j]$$

où $\mathcal{M}$ et $\mathcal{C}$ sont les ensembles de contraintes must-link et cannot-link, l_i désigne le label de cluster du point x_i et w_{ij} , $\bar{w}_{ij}$ modulent l'importance des contraintes par rapport à la distorsion géométrique. L'article étudié utilise un poids unique w tuné sur la partition de validation de chaque dataset.

La qualité des contraintes dépend fortement de la stratégie de sélection des paires. L'algorithme **Explore-Consolidate** (Basu et al., 2004) structure cette sélection en deux phases : une phase *Explore* qui identifie un point représentatif de chaque cluster en sélectionnant itérativement le point le plus éloigné de tous ceux déjà choisis, puis une phase *Consolidate* qui sollicite l'oracle sur les paires proches aux frontières entre clusters pour affiner le partitionnement. Cette stratégie maximise l'information extraite d'un nombre limité de requêtes et c'est celle que les auteurs reprennent pour leur pseudo-oracle LLM sur les datasets de clustering d'intentions.

Zhang, Basu et Davidson (2019) ont proposé d'intégrer les contraintes de paires dans des architectures deep learning, optimisant conjointement représentation et partitionnement sous contraintes. Ces approches surpassent PCKMeans sur certains benchmarks, en contrepartie d'un entraînement complet qui contredit le principe de simplicité *plug-in* revendiqué par Viswanathan et al., qui choisissent de conserver l'algorithme PCKMeans original.

1.2.3 La limite commune : le coût de l'annotation humaine

Dans une étude expérimentale combinant plusieurs modes d'interaction (contraintes de paires, split/merge, lock/refine) Coden et al. (2017) rapportent qu'il faut entre 20 et 100 interactions humaines pour qu'un algorithme de clustering produise un partitionnement jugé satisfaisant par l'expert même sur des corpus de taille modeste. Pour un corpus réel comportant des centaines de clusters, ce volume de feedback devient rapidement prohibitif. Viswanathan et al. (2024) répondent à cette limite en remplaçant l'oracle humain par une instance de GPT-3.5-turbo, ce qui leur permet de collecter 20,000 contraintes par dataset, soit deux à trois ordres de grandeur de plus qu'un annotateur humain à budget équivalent¹.

1.3 Les LLMs comme oracles, simulateurs et guides du clustering

L'arrivée des grands modèles de langage instruction-tunés a ouvert une troisième famille de méthodes plus récente que les deux précédentes et c'est dans cette famille que Viswanathan et al. (2024) s'inscrivent directement. Contrairement au clustering semi-supervisé classique qui mobilise un expert humain, ces approches exploitent un LLM comme source de supervision synthétique : oracle de décision, simulateur d'annotateur ou guide de structuration. Cette section retrace les fondements conceptuels de ce déplacement (1.3.1), cartographie les travaux appliqués au clustering de texte (1.3.2), puis situe précisément la contribution de l'article étudié (1.3.3).

1.3.1 Les LLMs comme simulateurs et évaluateurs : un cadre conceptuel

L'idée de substituer un LLM à un annotateur humain précède son application au clustering. Plusieurs travaux ont établi que les LLMs peuvent simuler des décisions individuelles (*homo silicus*, Horton, 2023), soutenir un rôle d'acteur rationnel sur la durée (*generative agents*, Park et al., 2023), noter la qualité de sorties textuelles de manière corrélée au jugement humain (GPTScore, Fu et al., 2023) et même surpasser des annotateurs crowd-sourcés sur des tâches de classification de texte (Gilardi, Alizadeh et Kubli, 2023).

Ces quatre contributions sur la simulation, le rôle, l'évaluation et l'annotation constituent le socle du *pseudo-oracle LLM* : un acteur artificiel dont les jugements restent suffisamment proches de la vérité terrain pour réduire le coût de supervision. C'est ce cadre que Viswanathan et al. mobilisent pour remplacer l'annotateur humain du PCKMeans original par une instance de GPT-3.5-turbo.

1.3.2 Panorama des LLMs appliqués au clustering de texte

L'application de ce cadre au clustering s'est densifiée entre 2023 et 2026. Les approches ainsi obtenues peuvent être organisées selon le **degré d'intégration** du LLM dans le pipeline : depuis des rôles purement auxiliaires jusqu'à des rôles de *clusterer* direct.

ClusterLLM (Zhang, Wang & Shang, 2023) est le concurrent historique le plus proche de l'article étudié. Les auteurs interrogent un LLM instruction-tuné sur des triplets durs (un ancre et deux candidats) afin d'obtenir un signal de préférence, puis utilisent ce signal pour affiner un encodeur plus petit. Un second mécanisme sonde la granularité de clustering via des questions par paires. L'originalité tient dans la parcimonie : environ 1,024 appels LLM suffisent par

¹L'écart entre le volume de contraintes généré par le pseudo-oracle LLM (20 000 paires par dataset) et l'estimation de Coden et al. (2017) sur le coût d'annotation humaine est au cœur de l'argument économique de Viswanathan et al., repris dans la section 1.3.3.

dataset. La méthode consiste donc à fine-tuner l’encodeur via retour LLM alors que Viswanathan et al. conservent l’encodeur gelé et injectent le signal LLM par d’autres canaux : enrichissement de features (keyphrases), contraintes dans la fonction objectif (PCKMeans) ou post-correction d’assignations. Ce choix de conception rend l’approche Keyphrase Clustering directement transposable à tout pipeline existant, là où ClusterLLM produit un encodeur spécialisé mais non réutilisable sans réentraînement².

IDAS (De Raedt et al., 2023) partage la philosophie d’intervention en amont mais reformule chaque énoncé via résumé abstraktif plutôt que d’enrichir les embeddings avec des keyphrases. Les deux méthodes démontrent que la valeur ajoutée du LLM réside dans la reformulation sémantique de l’entrée plus que dans une guidance fine du processus de clustering.

ZeroDL (Jo et al., 2025) supprime complètement l’encodeur externe. Le LLM est d’abord interrogé en zero-shot sur un sous-échantillon pour produire des descriptions ouvertes, ces descriptions sont agrégées en méta-information sur les classes latentes puis cette méta-information est réinjectée dans un nouveau prompt pour classer chaque texte. Le clustering devient une tâche entièrement texte-à-texte : là où Viswanathan et al. utilisent le LLM comme aide à un clusterer classique, ZeroDL en fait le clusterer lui-même.

Dial-In LLM (Hong et al., 2024) propose un framework itératif pour le clustering d’intentions et critique directement Viswanathan et al. (2024) qu’ils qualifient d’intégration superficielle. Ils rapportent que l’extension par phrases clés améliore le NMI mais peut distordre le sens original des énoncés, conduisant à un score de *goodness* inférieur au modèle de base³. **LLMEdgeRefine** (Feng et al., 2024) se concentre sur le raffinement des points frontières par LLM limitant donc les appels aux seuls cas ambigus.

Huang & He (2025) reformulent le clustering comme un problème de classification en deux étapes (génération de labels par le LLM puis classification des instances) illustrant un dépassement conceptuel de la dépendance aux encodeurs externes.

LUMI (Lin, Hasibi & Verberne, 2025) critique explicitement Viswanathan et al. sur deux points : le bruit des pseudo-labels issus d’un étiquetage en une passe et la restriction des décisions à des choix binaires qui ignorent les degrés continus de similarité. LUMI propose en réponse un mécanisme de pooling basé sur des ensembles de pseudo-labels partagés et sur des poids de similarité continue, méthode *training-free* et *label-free* qui obtient des résultats compétitifs sans nécessiter l’estimation préalable du nombre de clusters. Cette critique est particulièrement pertinente pour notre lecture car elle cible précisément le mécanisme de pseudo-oracle pairwise utilisé par Viswanathan et al. au stade *pendant* du clustering.

1.3.3 Positionnement de Viswanathan et al. (2024) dans ce paysage

L’apport spécifique de l’article étudié tient en trois points. Premièrement, c’est la première contribution à adopter une vue *systémique* du pipeline de clustering en articulant trois stades (avant, pendant, après) et en comparant empiriquement leur efficacité relative, cadre repris *a posteriori* par plusieurs travaux ultérieurs. Deuxièmement, l’approche revendique une simplicité méthodologique : aucune phase de fine-tuning, aucun réentraînement d’encodeur, aucun réglage spécifique au dataset. Cela rend la méthode *plug-in*, c’est-à-dire compatible avec n’importe quel encodeur gelé ou n’importe quel clusterer au prix auto-reconnu par les auteurs d’un nombre de requêtes LLM supérieur à celui de SCCL ou ClusterLLM⁴. Troisièmement, l’article établit empiriquement qu’à budget constant un pseudo-oracle LLM fournit environ 28 fois plus de labels qu’un annotateur humain, ouvrant un argument économique quantifié en faveur des approches synthétiques.

Viswanathan et al. occupent donc une position intermédiaire : orientés vers l’intégration légère du LLM à la manière de ClusterLLM et IDAS mais conceptuellement en transition vers les approches plus intégrées (Hong et al., Huang & He) qui utiliseront le LLM comme algorithme de clustering à part entière et non plus comme simple aide. Cette position explique à la fois l’influence de l’article comme point de référence citationnel et la nature des critiques qui lui sont adressées dans la littérature postérieure.

²Viswanathan et al. reconnaissent eux-mêmes (section 5.4 de leur article) que leur méthode requiert davantage de requêtes LLM que SCCL ou ClusterLLM.

³L’article original ne propose aucune évaluation qualitative de fidélité sémantique entre énoncé d’entrée et keyphrases générées. L’observation empirique de Hong et al. (2024) selon laquelle les keyphrases peuvent distordre le sens original corrobore une question laissée ouverte par l’article, qui pourra être investiguée dans nos évaluations complémentaires.

⁴En croisant les chiffres auto-rapportés par Viswanathan et al. (gains de 0,1 à 5,2 points pour la post-correction, section 5.3) avec le coût d’API documenté dans leur Table 5, on observe que le stade *après* est le maillon faible des trois stades. Le rapport coût-bénéfice est inégal : le *avant* (keyphrases) est le plus rentable, le *pendant* (PCKMeans pseudo-oracle) offre des gains intermédiaires, le *après* est quasi-nul en moyenne. Cette analyse suggère que le stade *pendant* est le plus susceptible d’améliorations méthodologiques, un diagnostic corroboré par Lin et al. (2025) sur le mécanisme des pseudo-labels binaires.

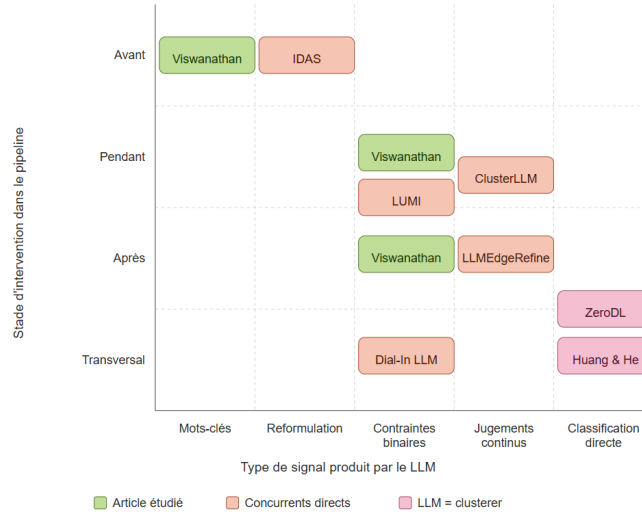


Fig. 1.3.1: Matrice de positionnement des approches guidées par LLM selon le type de signal LLM utilisé (abscisse) et le stade d'intervention dans le pipeline de clustering (ordonnée).

1.4 Synthèse et positionnement de l'état de l'art

Les trois familles présentées dans cet état de l'art se distinguent par les compromis qu'elles établissent entre qualité du partitionnement, coût computationnel et effort demandé à l'utilisateur. Les méthodes classiques (section 1.1) n'exigent aucune supervision mais restent limitées par la pauvreté sémantique des représentations creuses. Les approches de clustering profond et contrastif améliorent la qualité en apprenant des représentations spécialisées, ce qui implique un entraînement coûteux et une perte de transférabilité. Le clustering semi-supervisé (section 1.2) résout la sous-spécification en injectant des contraintes utilisateur mais transfère le coût vers l'expert humain. Les approches guidées par LLM (section 1.3) substituent un pseudo-oracle automatique à cet expert, ce qui rend possible un volume de supervision beaucoup plus élevé pour un coût marginal modeste, en contrepartie d'une dépendance à un service externe et d'une incertitude sur la fiabilité du jugement automatique.

Viswanathan et al. (2024) se positionnent dans cette dernière famille avec une stratégie spécifique : plutôt que de proposer une méthode unique, ils explorent systématiquement trois points d'intervention du LLM dans le pipeline de clustering. Leur contribution est moins l'invention d'une technique nouvelle que la cartographie empirique de ces trois stades et l'identification du plus rentable. Le schéma ci-après (cf. Fig.~1.4.1) synthétise la taxonomie des approches couvertes par cet état de l'art et situe l'article étudié dans cette structure.

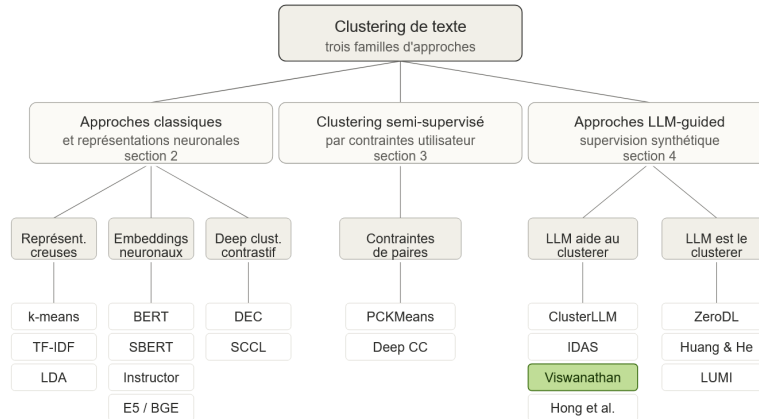


Fig. 1.4.1: Taxonomie des approches couvertes par l'état de l'art. La racine désigne la tâche, le premier niveau les trois familles (sections 1.1, 1.2, 1.3), le deuxième niveau les sous-familles conceptuelles, le troisième niveau les méthodes représentatives. Deep CC : Deep Constrained Clustering (Zhang, Basu & Davidson, 2019). SBERT : Sentence-BERT.

2 Description de l’approche

Les auteurs proposent d’intégrer un LLM à trois stades du pipeline de clustering de texte semi-supervisé. **Avant le clustering**, le LLM enrichit la représentation de chaque document en générant des keyphrases qui sont encodées puis concaténées à l’embedding original. **Pendant le clustering**, le LLM remplace l’oracle humain du PCKMeans en classifiant automatiquement des paires de points comme devant ou ne devant pas appartenir au même cluster. **Après le clustering**, le LLM corrige les assignations à faible confiance en réévaluant leur appartenance parmi les clusters candidats les plus proches. Cette section présente d’abord les tâches et datasets sur lesquels ces méthodes sont évaluées (2.1), puis décrit chacune des trois méthodes (2.2), détaille les baselines et le protocole expérimental retenus par les auteurs (2.3) et enfin rapporte les résultats présentés dans l’article (2.4).

2.1 Tâches et datasets

L’article évalue ses méthodes sur deux tâches du clustering de texte qui mobilisent chacune des benchmarks et des métriques propres : la **canonicalisation d’entités** issues de l’extraction d’information ouverte (2.1.1) et le **clustering d’intentions** dans les systèmes conversationnels (2.1.2).

2.1.1 Canonicalisation d’entités

La première tâche consiste à regrouper des mentions textuelles qui désignent la même entité du monde réel. Les expressions *Joe Biden*, *President Biden* et *the 46th U.S. President* doivent être placées dans le même cluster bien qu’elles diffèrent en surface. Contrairement à l’entity linking, aucune base de connaissances de référence n’est supposée : les clusters doivent être produits à partir des mentions seules.

Pour représenter chaque entité, les auteurs s’appuient sur l’architecture CMVC qui combine deux vues complémentaires. La *fact view* encode la position de l’entité dans le graphe de connaissances Open IE via un modèle TransE : chaque nœud est représenté par un vecteur dense appris de telle sorte que les relations entre nœuds correspondent à des translations dans l’espace vectoriel. La *context view* encode le contexte textuel des mentions via BERT, en agrégeant les embeddings des phrases où l’entité apparaît. Les auteurs proposent une simplification de CMVC : au lieu du clustering alterné entre vues proposé dans le papier original, ils concatènent simplement les deux représentations et appliquent k-means++ sur l’espace concaténé. Ils rapportent des performances comparables à la méthode originale.

Un point important concerne la granularité du clustering. L’article ne regroupe pas les mentions individuelles mais les formes de surface⁵ uniques : toutes les mentions de “*Marseille*” dans différentes phrases sont d’abord regroupées sous la même forme de surface puis c’est cette forme qui est assignée à un cluster. Cette procédure introduit des erreurs irréductibles pour les formes polysémiques (“*Marseille*” peut désigner la ville ou le club sportif) et les auteurs sont les premiers à signaler cette limitation en rapportant les performances optimales atteignables sous cette hypothèse.

Deux datasets sont utilisés. **OPIEC59k** contient 22 000 mentions (2 138 formes de surface uniques) regroupées en 490 clusters avec un nombre de clusters fixé à 490 via la méthode Log-Jump. **ReVerb45k** contient 15 500 mentions (12 295 formes de surface uniques) regroupées en environ 6 700 clusters, ce qui en fait un benchmark considérablement plus difficile en raison du grand nombre de petits clusters. Les résultats sont évalués selon trois scores F1 complémentaires : la **macro F1** mesure la cohérence au niveau des clusters (tous les éléments d’un cluster prédit sont-ils dans le même cluster de référence ?), la **micro F1** au niveau des points (chaque point est-il assigné au même cluster que la majorité de ses voisins de référence ?) et la **pairwise F1** au niveau des paires (les paires prédites comme liées le sont-elles effectivement ?).

2.1.2 Clustering d’intentions et de tweets

La seconde tâche consiste à regrouper des énoncés courts selon l’intention qu’ils expriment. Les énoncés “*Comment puis-je localiser ma carte ?*” et “*Où se trouve ma carte bancaire ?*” doivent être placés dans le même cluster. Les frontières entre intentions peuvent être très fines : “*je veux annuler mon vol*” et “*je veux modifier mon vol*” partagent presque tout leur vocabulaire mais expriment des intentions distinctes.

Trois datasets sont utilisés. **Bank77** contient 3 080 requêtes adressées à un assistant bancaire, réparties en 77 catégories d’intention à granularité fine au sein d’un domaine unique. **CLINC** contient 4 500 requêtes adressées à un assistant conversationnel généraliste, réparties en 150 catégories couvrant 10 domaines distincts. **Tweet** contient 2 472 tweets répartis en 89 catégories thématiques. Pour Bank77 et CLINC, les auteurs encodent chaque requête avec Instructor-large en utilisant le prompt “**Represent utterances for intent classification**”. Pour Tweet, ils utilisent DistilBERT fine-tuné pour la similarité de phrases. Les résultats sont évalués par l’**accuracy** (après alignement optimal par algorithme hongrois) et la **Normalized Mutual Information (NMI)**.

⁵Une *forme de surface* (*surface form*) désigne la chaîne de caractères exacte par laquelle une entité est mentionnée dans le texte. L’entité *Joe Biden* peut ainsi apparaître sous les formes de surface “Joe Biden”, “President Biden” ou “the 46th U.S. President”. L’article opère le clustering au niveau des formes de surface uniques et non des mentions individuelles.

Le tableau ci-après récapitule les caractéristiques des cinq datasets.

Dataset	Tâche	Exemples	Clusters	Encodeur	Métriques
OPIEC59k	Canonicalisation	22 000	490	CMVC	F1 (macro, micro, pair)
ReVerb45k	Canonicalisation	15 500	~6 700	CMVC	F1 (macro, micro, pair)
Bank77	Intentions (banque)	3 080	77	Instructor	Acc, NMI
CLINC	Intentions (multi-dom.)	4 500	150	Instructor	Acc, NMI
Tweet	Clustering thématique	2 472	89	DistilBERT	Acc, NMI

Fig. 2.1.1: Synthèse des cinq datasets utilisés par Viswanathan et al. (2024). La colonne Encodeur indique le modèle utilisé comme baseline pour chaque dataset.

2.2 Les trois méthodes proposées

Les trois méthodes partagent un même principe : l’interaction entre l’utilisateur et le clustering passe par un prompt rédigé par l’utilisateur et soumis à un LLM (GPT-3.5-turbo-0301 dans toutes les expériences). Ce prompt comprend une instruction spécifique à la tâche suivie de quatre démonstrations tirées des labels du jeu de test. Le LLM produit alors un signal de supervision synthétique qui est injecté à différents stades du pipeline.

2.2.1 Avant le clustering : enrichissement par keyphrases

La première méthode intervient avant toute opération de clustering en enrichissant la représentation de chaque document. L’intuition est la suivante : un expert sait quels aspects d’un document sont pertinents pour le regroupement souhaité (le sujet, l’intention, l’entité désignée), mais un encodeur généraliste n’a pas accès à cette information. Plutôt que de fine-tuner l’encodeur, les auteurs demandent au LLM de rendre ces aspects explicites sous forme de keyphrases.

La procédure se déroule en trois temps. D’abord, chaque document est soumis au LLM avec un prompt qui décrit la tâche de clustering et demande de générer une liste de keyphrases au format JSON. Par exemple, pour le clustering d’intentions bancaires, le prompt commence par “*I am trying to cluster online banking queries based on whether they express the same intent. For each query, generate a comprehensive set of keyphrases that could describe its intent, as a JSON-formatted list.*”(section 2.1 de l’article), suivi de quatre exemples de paires requête-keyphrases. Pour la canonicalisation d’entités, le prompt demande plutôt de générer des noms alternatifs pour une entité donnée en s’appuyant sur des phrases de contexte fournies en entrée.

Ensuite, les keyphrases générées sont encodées en un vecteur unique par le même encodeur que celui utilisé pour le texte original. Ce choix permet de découpler la connaissance apportée par le LLM (le contenu des keyphrases) du bénéfice d’un meilleur encodeur. Une exception concerne la canonicalisation d’entités : l’encodeur BERT ayant été spécialisé pour les phrases Wikipedia dans l’architecture CMVC, les auteurs utilisent DistilBERT pour encoder les keyphrases afin d’éviter un biais de domaine (note 2, p. 2 de l’article).

Enfin, le vecteur de keyphrases est concaténé au vecteur du document original pour produire une représentation enrichie de dimension doublée. Le clustering par k-means++ est alors appliqué sur ces vecteurs concaténés. Le coût de cette méthode est d’un appel LLM par document, ce qui la rend la moins coûteuse des trois approches pour les datasets de taille modeste (environ 1 à 2 dollars par dataset selon la Table 5 de l’article pour les datasets d’intentions et environ 10 dollars pour ReVerb45k qui contient plus de 12 000 formes de surface uniques).

2.2.2 Pendant le clustering : PCKMeans avec pseudo-oracle LLM

La deuxième méthode intervient pendant le clustering en fournissant des contraintes de paires à l’algorithme PCKMeans, dont la fonction objectif et le mécanisme ont été décrits en section 1.2.2. L’innovation consiste à remplacer l’oracle humain par un LLM qui classe automatiquement les paires.

Sélection des paires. Les auteurs adoptent deux stratégies selon la tâche. Pour le clustering d’intentions (Bank77, CLINC, Tweet) ils adaptent l’algorithme Explore-Consolidate décrit en section 1.2.2 : la phase Explore collecte des paires diversifiées dans l’espace des embeddings pour identifier les frontières entre clusters puis la phase Consolidate collecte des paires proches pour affiner ces frontières. Pour la canonicalisation d’entités où le nombre de clusters est très élevé (490 à 6 700) et la probabilité qu’une paire aléatoire soit must-link est donc très faible, les auteurs adoptent une stratégie plus simple : ils identifient les paires de points les plus proches dans l’espace des embeddings qui appartiennent à des formes de surface distinctes, ce qui maximise la probabilité d’obtenir des paires informatives.

Classification des paires par le LLM. Chaque paire sélectionnée est soumise au LLM avec un prompt spécifique à la tâche. Pour la canonicalisation, le prompt fournit les deux formes de surface accompagnées de phrases de contexte Wikipedia et demande si elles réfèrent à la même entité. Pour le clustering d'intentions, le prompt fournit les deux requêtes utilisateur et demande si elles expriment la même intention. Dans les deux cas, le prompt contient quatre démonstrations de paires avec leur label (must-link ou cannot-link), tirées des labels du jeu de test. Les prompts complets sont fournis en Appendice A de l'article.

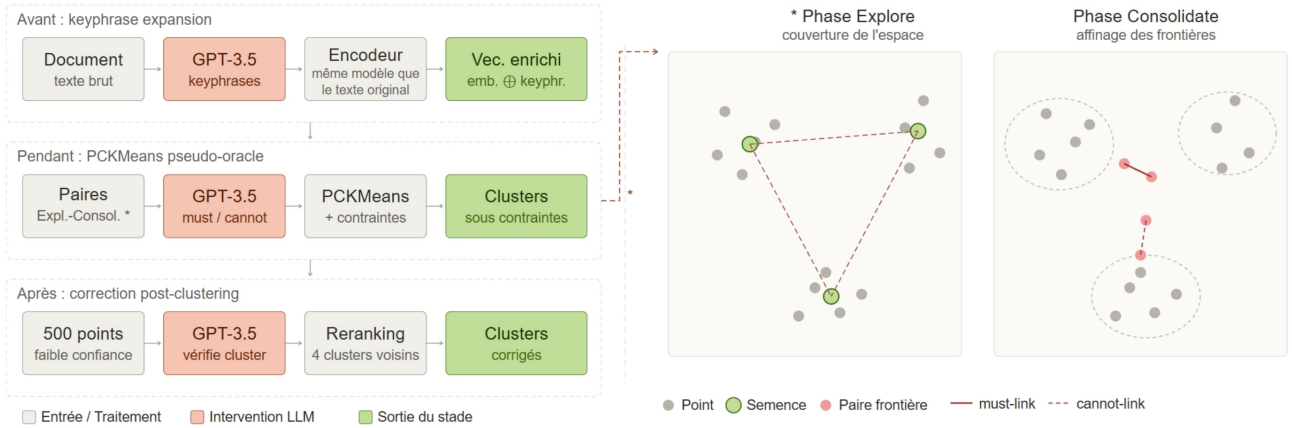
Injection des contraintes. Les paires classifiées sont injectées dans PCKMeans sous forme de contraintes must-link et cannot-link. L'hyperparamètre w qui pondère l'importance des contraintes par rapport à la distorsion géométrique dans la fonction objectif est tuné sur le split de validation de chaque dataset. Les auteurs collectent 20 000 paires par dataset, ce qui fait de cette méthode la plus coûteuse des trois en nombre d'appels LLM (environ 10 à 42 dollars par dataset selon la Table 5 de l'article).

2.2.3 Après le clustering : correction des assignations à faible confiance

La troisième méthode intervient après qu'un clustering initial a été produit et vise à en corriger localement les erreurs. Les auteurs identifient les $k = 500$ points dont l'assignation est la moins fiable, définis comme ceux ayant la plus faible marge entre la distance au centroïde du cluster assigné et la distance au centroïde du deuxième cluster le plus proche.

Pour chaque point à faible confiance, la procédure se déroule en deux temps. D'abord, le LLM évalue si ce point est correctement lié à son cluster actuel. Pour cela, le cluster est représenté textuellement par les entités les plus proches de son centroïde dans l'espace des embeddings et le LLM est interrogé sur la relation pairwise entre le point et ces représentants. Si le LLM juge que le point n'appartient pas au cluster actuel, les quatre clusters les plus proches dans l'espace des embeddings sont considérés comme candidats au reclassement. Le LLM évalue alors séquentiellement la relation du point avec les représentants de chaque cluster candidat en commençant par le plus proche. Dès qu'une réponse positive est obtenue, le point est réassigné à ce nouveau cluster. Si aucune correspondance n'est trouvée parmi les candidats, l'assignation originale est conservée.

Cette méthode utilise le même pseudo-oracle pairwise que la méthode PCKMeans (section 2.2.2) mais l'applique de manière ciblée sur un sous-ensemble restreint de points. Le coût en requêtes LLM est intermédiaire (environ 3 à 13 dollars par dataset) mais le gain empirique rapporté est limité : entre 0,1 et 5,2 points d'amélioration selon les métriques et les datasets, comme le détaille la section 2.4.



*Fig. 2.2.1: Pipeline détaillé des trois stades d'intervention du LLM dans le clustering (gauche) et détail de l'algorithme Explore-Consolidate utilisé pour la sélection des paires au stade Pendant (droite, marqué par *). Adapté de Viswanathan et al. (2024).*

2.3 Baselines et protocole expérimental

Les auteurs comparent leurs trois méthodes à plusieurs baselines qui permettent d'isoler l'apport du LLM à chaque stade du pipeline.

Baselines pour la canonicalisation. La baseline principale est CMVC, dans la version simplifiée décrite en section 2.1.1 : les vues fact et context sont concaténées puis partitionnées par k-means++. Cette simplification constitue elle-même une baseline puisqu'elle supprime le clustering alterné de CMVC original tout en conservant des performances comparables. Le nombre de clusters est estimé automatiquement par la méthode Log-Jump, ce qui évite de supposer le nombre de clusters connu à l'avance.

Baselines pour le clustering d'intentions. Deux familles de baselines sont mobilisées. La première est k-means++ appliqué directement sur les embeddings bruts (Instructor pour Bank77 et CLINC, DistilBERT pour Tweet) sans aucune intervention du LLM. La seconde regroupe les méthodes de clustering profond qui apprennent conjointement représentation et partitionnement : SCCL et ClusterLLM, déjà présentées en section 1.1.3 de l'état de l'art. Pour ces datasets, le nombre de clusters K est fixé à sa valeur réelle (77, 150, 89), conformément au protocole standard de la littérature.

Protocole expérimental. Chaque expérience est répétée sur 5 seeds aléatoires et les auteurs rapportent la moyenne des métriques. Pour la méthode PCKMeans pseudo-oracle (section 2.2.2), l'hyperparamètre w est tuné sur un split de validation distinct du jeu de test. Les quatre démonstrations incluses dans les prompts LLM sont tirées des labels du jeu de test, ce qui correspond à un régime few-shot où l'utilisateur fournit un nombre minimal d'exemples pour spécifier son intention de clustering. Le modèle LLM utilisé dans toutes les expériences est GPT-3.5-turbo-0301 (version datée de mars 2023, figée par OpenAI), ce qui garantit la reproductibilité des appels API tant que cette version reste accessible.

2.4 Résultats rapportés par les auteurs

Les résultats principaux sont présentés dans les Tables 1 et 2 de l'article. Trois constats se dégagent.

Le stade Avant (keyphrase expansion) est le plus performant en moyenne. Sur les cinq datasets, le keyphrase clustering améliore les scores par rapport à la baseline k-means++ sur trois d'entre eux (OPIEC59k, Bank77, CLINC) et produit des résultats comparables sur les deux autres (ReVerb45k, Tweet). Sur CLINC, le gain atteint +5,8 points d'accuracy et +3,4 points de NMI par rapport à la baseline Instructor. Ce stade est aussi le moins coûteux en appels LLM.

Le stade Pendant (PCKMeans pseudo-oracle) est compétitif mais plus variable. Il obtient le meilleur score absolu sur OPIEC59k en pairwise F1 et sur Bank77 en accuracy mais sous-performe le keyphrase clustering sur CLINC et Tweet. Sur ReVerb45k, les contraintes LLM n'améliorent pas significativement les résultats, ce que les auteurs attribuent au grand nombre de clusters qui rend la sélection de paires informatives difficile. Les 20 000 contraintes collectées représentent un coût d'API nettement supérieur au stade Avant.

Le stade Après (correction post-clustering) apporte des gains marginaux. Les améliorations varient entre 0,1 et 5,2 points selon les datasets et les métriques (Table 2 de l'article). Les auteurs rapportent eux-mêmes que ce stade est le moins rentable des trois en termes de rapport coût-bénéfice.

Ablation sur la structure des prompts. La Table 3 de l'article isole l'effet des deux composantes du prompt : l'instruction seule (sans démonstrations) et les démonstrations seules (sans instruction). Sur Bank77, l'instruction seule produit un score d'accuracy de 72,4 contre 75,2 avec le prompt complet et les démonstrations seules atteignent 73,8. La combinaison des deux composantes est systématiquement supérieure à chacune prise isolément mais l'instruction apporte la plus grande part du gain.

Coût d'API. La Table 5 de l'article documente le coût en dollars de chaque méthode. Le keyphrase clustering coûte entre 1 et 10 dollars par dataset, le PCKMeans pseudo-oracle entre 10 et 42 dollars et la correction post-clustering entre 3 et 13 dollars. Rapporté au gain en performance, le stade Avant offre le meilleur retour sur investissement. Les auteurs comparent par ailleurs le coût du pseudo-oracle LLM à celui d'un annotateur humain et estiment qu'à budget constant le LLM fournit environ 28 fois plus de contraintes.

L'approche des auteurs repose donc sur un principe commun aux trois stades : substituer un LLM à un expert humain pour produire un signal de supervision synthétique, sans modifier l'encodeur ni l'algorithme de clustering sous-jacent. Les résultats rapportés montrent que les trois stades ne contribuent pas de manière égale : le stade Avant (keyphrase expansion) offre le meilleur rapport coût-performance, le stade Pendant (PCKMeans pseudo-oracle) apporte des gains compétitifs mais à un coût sensiblement plus élevé et le stade Après (correction post-clustering) produit des améliorations marginales. La section suivante examine de manière critique les forces et les limites de cette approche ainsi que les choix méthodologiques qui conditionnent la portée des résultats.

3 Analyse critique

L'article de Viswanathan et al. repose sur une idée méthodologiquement claire : plutôt que de proposer un algorithme de clustering entièrement nouveau, les auteurs injectent un signal LLM à trois endroits naturels d'un pipeline existant. Cette section en examine d'abord les points forts (3.1), puis les limitations (3.2), les fragilités du protocole expérimental et des évaluations proposées (3.3) et enfin la structure des prompts utilisés (3.4).

3.1 Points forts de l’approche

3.1.1 Un pipeline modulaire

L’article se distingue par la clarté de son cadre conceptuel. La structuration du problème autour d’un pipeline en trois stades (avant, pendant et après le clustering) constitue un cadre d’analyse systématique qui dépasse la simple proposition d’une nouvelle méthode isolée. Les auteurs proposent une architecture modulaire dans laquelle un LLM peut être injecté à différents niveaux d’un pipeline existant sans en modifier la structure fondamentale. Cette modularité constitue en soi une contribution méthodologique qui permet d’évaluer indépendamment l’apport du LLM à chaque stade et d’identifier où son intervention est la plus décisive.

3.1.2 Enrichissement sémantique

La méthode d’enrichissement des représentations par génération de keyphrases constitue l’un des apports les plus convaincants de l’article. Les keyphrases générées sont encodées avec le même encodeur que les données brutes et les deux représentations sont ensuite concaténées. Ce choix de conception relève d’une hygiène méthodologique rigoureuse : en gardant l’encodeur constant des deux côtés, tout différentiel de performance observé est attribuable à l’information sémantique injectée par le LLM et non à un changement d’architecture. Si un encodeur plus performant avait été utilisé pour les keyphrases, il aurait été impossible de distinguer l’apport informationnel du LLM du bénéfice d’un meilleur encodeur. La fusion par concaténation plutôt que par remplacement relève par ailleurs d’une logique prudente : en conservant l’embedding original à côté des keyphrases, la méthode limite la dégradation si les keyphrases sont bruitées, hallucinées ou incomplètes, l’embedding d’origine servant de filet de sécurité dans le vecteur final.

Une autre propriété mise en avant par les auteurs est le caractère plug-in de la méthode, superposable à n’importe quel pipeline existant sans modification algorithmique. Une fois les keyphrases générées, l’encodeur peut être gelé et aucun fine-tuning n’est requis. Cette propriété répond à l’un des défis récurrents des méthodes de deep learning appliquées au clustering : la dépendance à un réentraînement coûteux pour chaque nouvelle tâche. Elle distingue fondamentalement l’approche proposée de SCCL et ClusterLLM dont les encodeurs spécialisés atteignent de bonnes performances dans leur contexte d’entraînement mais nécessitent un nouveau cycle de fine-tuning dès que le contexte change. L’approche découple ainsi la génération de connaissance sémantique, assurée par le LLM, de l’encodage, ce qui lui confère une flexibilité structurelle que les méthodes basées sur l’apprentissage profond supervisé peinent à offrir.

3.1.3 Supervision via pseudo-oracle

L’utilisation du LLM comme pseudo-oracle pour générer des contraintes must-link / cannot-link représente une application originale de la littérature sur les LLMs comme simulateurs de jugements humains. Par rapport à un oracle humain, le LLM permet de passer à l’échelle : là où un annotateur humain ne peut traiter qu’un nombre limité de paires, le LLM peut en annoter un grand nombre de façon automatisée, tout en s’appuyant sur des algorithmes de sélection conçus pour maximiser l’utilité informationnelle des contraintes générées. L’algorithme Explore-Consolidate en est l’exemple central : il cible prioritairement les paires situées près des frontières de décision, là où l’incertitude est maximale et où une contrainte pairwise a le plus d’impact sur la structure finale des clusters.

Les auteurs font par ailleurs preuve d’adaptabilité en reconnaissant que certaines tâches comme la canonicalisation d’entités, caractérisée par un nombre élevé de clusters et peu de points par cluster, nécessitent une stratégie de sélection différente. Cette capacité à ajuster la méthode selon le contexte témoigne d’une conscience lucide des limites de l’approche générale ce qui renforce la crédibilité de la contribution.

À cela s’ajoute la transparence du formalisme PCKMeans dont la fonction objectif intègre des pénalités pondérées pour les violations de contraintes, rendant le compromis entre fidélité géométrique et respect des contraintes explicitement contrôlable.

3.1.4 Correction ciblée des assignments

La phase de post-correction par LLM repose sur la notion de margin. Le LLM reclassifie les points dont les distances aux centroïdes les plus proches sont les plus similaires et qui sont identifiés comme ambigus. Ce transfert du concept de margin, issu des SVM et de l’active learning, vers le clustering constitue une transposition conceptuelle non triviale où la margin devient un indicateur d’incertitude d’appartenance.

Cette étape présente un intérêt conceptuel qui dépasse ses résultats empiriques, lesquels restent limités selon les propres conclusions des auteurs. Plutôt que de ne retenir que les contributions les plus performantes, les auteurs choisissent d’exposer une piste dont l’apport pratique est modeste mais dont la logique est cohérente avec le cadre général de l’approche. Cette transparence renforce la crédibilité de l’ensemble de la contribution. On peut également noter que cette phase réutilise le même pseudo-oracle pairwise que le stade Pendant, ce qui maintient la cohérence du framework et évite la prolifération de prompts distincts pour chaque stade.

3.2 Limitations identifiées

3.2.1 Des choix non validés et un coût éludé

Un premier biais méthodologique concerne la canonicalisation d’entités : les auteurs substituent l’encodeur BERT fine-tuné par CMVC par un DistilBERT généraliste pour encoder les keyphrases. Cette asymétrie rompt la règle méthodologique que les auteurs s’imposent eux-mêmes pour les autres tâches, à savoir utiliser le même encodeur pour le texte original et pour les keyphrases. La justification avancée (le BERT de CMVC est spécialisé Wikipédia et encoderait mal des keyphrases génériques, note~2, p.~2 de l’article) est raisonnable mais elle signifie que pour cette tâche, les améliorations observées ne peuvent plus être strictement attribuées à l’enrichissement par keyphrases : une partie pourrait venir du changement d’encodeur sur la branche keyphrase.

Au-delà de ce biais, les jeux de données sur lesquels la méthode est évaluée partagent une caractéristique qui pèse sur la portée des résultats : ils sont tous composés de textes courts. Le clustering de textes courts est un régime spécifique où la pauvreté lexicale rend particulièrement précieuse toute forme d’enrichissement sémantique, créant ainsi les conditions idéales pour que le signal LLM se traduise en gain mesurable. Sur des documents longs, plusieurs effets contraires se manifesteraient. D’abord, l’enrichissement par keyphrases apporterait un signal proportionnellement plus faible, la représentation initiale étant déjà riche. Ensuite, le coût de l’approche augmenterait fortement : le nombre de tokens traités par le LLM croît linéairement avec la longueur des documents, ce qui déplacerait le point d’équilibre économique identifié par les auteurs.

Enfin, la variance liée au non-déterminisme du LLM n’est jamais caractérisée. À température strictement positive, GPT-3.5 peut produire des keyphrases différentes à chaque exécution. Or les écarts-types rapportés dans les tables sont calculés sur les seeds de k-means pas sur des régénérations du LLM. La sensibilité au wording du prompt n’est pas davantage testée alors que la littérature documente abondamment la fragilité des LLMs face aux reformulations mineures d’instructions. Cette absence de caractérisation de la variance LLM est particulièrement problématique pour la reproductibilité des résultats.

3.2.2 Fragilité du pseudo-oracle face au bruit de décision

La Figure 5 de l’article compare la trajectoire d’apprentissage de PCKMeans sous deux régimes de supervision sur OPIEC59k : un oracle humain parfait et le pseudo-oracle LLM. Alors que l’oracle parfait permet à la Macro F1 de croître quasi-linéairement jusqu’à 20,000 contraintes, la courbe LLM sature dès environ 5,000 contraintes et plafonne ensuite. Cette saturation traduit une propriété structurelle du pseudo-oracle : au-delà d’un certain volume, les décisions correctes sont noyées par un bruit de fond stable qui limite l’information exploitable par l’algorithme. Ce seuil effectif d’utilité n’est pourtant jamais discuté explicitement par les auteurs alors qu’il a une implication directe sur le coût de la méthode puisque les contraintes générées au-delà de ce seuil n’améliorent plus la qualité des clusters tout en continuant d’engager un coût API proportionnel. Le nombre de contraintes est par ailleurs fixé à 20,000 sans justification a priori. La justification n’apparaît qu’a posteriori dans la section 5.4 de l’article via l’argument économique, mais il s’agit d’un hyperparamètre majeur dont la valeur conditionne à la fois la performance et le coût de la méthode. Cette saturation, pressentie à partir de la Figure 5 de l’article, sera confirmée empiriquement dans nos évaluations complémentaires (§4.1, sous-section saturation du budget de contraintes).

Une seconde fragilité tient à la nature binaire des décisions demandées au LLM. Pour chaque paire (x_i, x_j) , le modèle doit répondre par oui ou par non, sans possibilité d’exprimer un degré de confiance. Cette discrétisation élimine une information que la représentation vectorielle sous-jacente conserve : deux paires classées must-link peuvent en réalité correspondre à des degrés très différents de proximité sémantique. Lin et al. (2025) critiquent explicitement ce choix et proposent en réponse un mécanisme de similarité continue, qu’ils démontrent empiriquement supérieur à budget d’API équivalent.

Un problème de cohérence logique se pose enfin au niveau des contraintes générées. Les contraintes PCKMeans sont souples (pénalisées dans la fonction objectif, non imposées) ce qui signifie que si le LLM génère des triplets transitivement incohérents, l’algorithme les absorbe silencieusement en privilégiant l’inertie géométrique. Ce comportement est d’autant plus problématique que le stade Pendant est précisément celui où la qualité du signal d’entrée est la plus critique : contrairement au stade Avant où une keyphrase imprécise est simplement diluée dans la représentation finale, une contrainte incohérente pendant l’optimisation perturbe directement la convergence de l’algorithme. La fréquence de ces incohérences transitives n’est jamais mesurée dans l’article alors qu’elle constituerait un indicateur précieux de la fiabilité effective du pseudo-oracle⁶

⁶Cette lacune fait précisément l’objet de notre évaluation complémentaire §4.2, dans laquelle nous instrumentons les décisions pairwise et quantifions le taux d’incohérences transitives par dataset.

3.2.3 Biais dans la sélection des baselines

Le choix des baselines conditionne fortement la lecture des résultats présentés en Tables 1 et 2 de l’article. Deux observations appellent une lecture prudente des gains rapportés.

Premièrement, aucune baseline de clustering semi-supervisé avec oracle humain à volume comparable n’est évaluée dans le régime des résultats principaux. Les auteurs produisent bien, en section 5.5 de l’article, une comparaison entre PCKMeans alimenté par un oracle LLM et PCKMeans alimenté par un oracle parfait (Figure 5), mais cet oracle parfait est simulé à partir des labels de vérité terrain et ne représente pas le coût réel d’un annotateur humain. Une vraie baseline humaine aurait permis de mesurer l’écart qualitatif entre un petit volume d’annotations humaines (typiquement 500 à 2,000 paires) et un grand volume d’annotations synthétiques. Ce décalage de volume est précisément ce qui rend la comparaison asymétrique.

Deuxièmement, la famille self-supervised learning est absente du dispositif comparatif. Des méthodes comme SimCSE (Gao et al., 2021) ou DiffCSE (Chuang et al., 2022) produisent des représentations textuelles denses sans aucune supervision explicite et pourraient constituer une alternative à Instructor comme encodeur de base sur Bank77 et CLINC. Cette absence est d’autant plus notable que l’article se présente comme une contribution plug-in compatible avec n’importe quel encodeur. Or Instructor est lui-même un encodeur supervisé fine-tuné sur des paires instruction-texte, ce qui introduit un biais de conception favorable à l’enrichissement par keyphrases, ces dernières étant précisément formulées comme des instructions. Tester la méthode avec un encodeur self-supervised aurait permis de dissocier la part d’amélioration imputable au LLM de celle imputable au choix de l’encodeur sous-jacent.

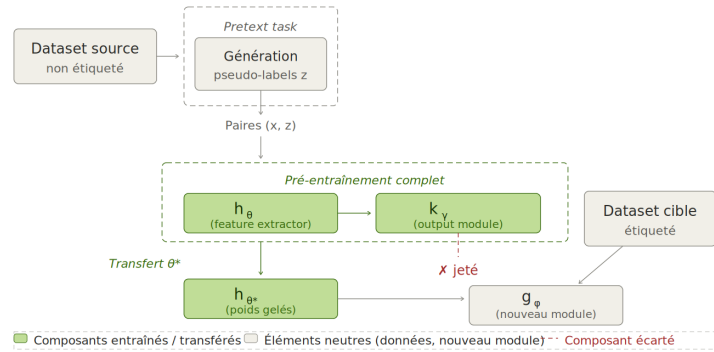


Fig. 3.2.1: Pipeline de l’apprentissage auto-supervisé. Un modèle est pré-entraîné sur des données non étiquetées via une tâche de génération, puis affiné pour une tâche cible supervisée.

3.2.4 Fragilités structurelles de la correction ciblée

Au-delà de ses gains empiriques modestes (0,1 à 5,2 points), la phase de correction post-clustering présente plusieurs limitations structurelles que l’article ne discute pas.

La procédure de reranking privilégie de fait la géométrie sur le jugement du LLM. Le pseudo-oracle interroge les quatre clusters candidats dans l’ordre de proximité au centroïde et retient le premier pour lequel il obtient une réponse positive. Un point pourrait recevoir un jugement favorable pour le deuxième et le quatrième cluster, mais l’algorithme ne retient que le deuxième puisqu’il est interrogé en premier. C’est donc l’ordre géométrique qui tranche ce qui contredit l’esprit d’une méthode censée laisser le LLM arbitrer. Cette observation motive la baseline géométrique que nous évaluons en §4.3 : si la géométrie tranche en pratique, une règle géométrique pure devrait reproduire l’essentiel du gain attribué au LLM.

La méthode souffre par ailleurs d’une incapacité structurelle à corriger les erreurs de granularité. Elle ne peut que réassigner des points entre clusters existants, sans possibilité de créer ou de fusionner des clusters. Si k-means a produit 77 clusters alors qu’il en fallait 80, aucune post-correction ne fera apparaître les 3 clusters manquants. Cette limite fondamentale n’est pas mentionnée dans l’article.

Enfin, la représentation d’un cluster par les points les plus proches de son centroïde est fragile lorsque le cluster est mal formé. Si le k-means initial a mélangé des classes, les représentants proches du centroïde peuvent eux-mêmes être hétérogènes et le LLM recevra une description contradictoire du cluster. Le risque de propagation d’erreur est alors élevé : un cluster mal formé sera mal représenté, ce qui conduira à des décisions de réassignation elles-mêmes erronées.

3.3 Analyse des évaluations proposées par les auteurs

3.3.1 Ablations et décomposition du signal utile

L’un des passages les plus instructifs de l’article est l’ablation présentée en Table 3. Sur OPIEC59k et CLINC, les auteurs comparent trois variantes du prompt Keyphrase Clustering : prompt complet (instruction + démonstrations), instruction seule (sans démonstrations) et démonstrations seules (sans instruction). Le résultat est remarquable : retirer les démonstrations ne coûte que 0,7 point d’accuracy sur CLINC et retirer l’instruction coûte 1,0 point. Les auteurs reconnaissent cette observation mais en tirent une conclusion minimale : l’instruction seule apporte déjà l’essentiel du gain.

Cette conclusion mérite d’être poussée plus loin. Si une instruction textuelle brève suffit à produire la majorité du signal utile, alors le cadrage few-shot du titre de l’article est susceptible d’induire en erreur : le mécanisme dominant n’est pas l’apprentissage par quelques exemples mais la spécification de tâche par instruction naturelle, un régime qui relève davantage du zero-shot prompting. Cette lecture suggère que le LLM mobilise principalement sa connaissance paramétrique préexistante plutôt qu’un véritable mécanisme d’adaptation en contexte, ce qui déplace la contribution réelle de l’article vers l’ingénierie de prompt plutôt que vers le few-shot learning au sens strict.

Une piste naturelle consisterait à remplacer le LLM généraliste par un encodeur plus petit conditionnable par instruction. La Table 3 montre cependant que cette substitution ne fonctionne pas : Instructor-XL conditionné avec le prompt complet de GPT-3.5 atteint seulement 70,8% d’accuracy, contre 79,4% pour GPT-3.5. L’écart persiste et les auteurs l’attribuent à la capacité des grands LLMs à traiter des prompts longs et détaillés. Si cette analyse est correcte, elle a des conséquences directes sur la reproductibilité : la méthode ne serait efficace qu’avec des LLMs d’une certaine capacité, ce qui réintroduit une dépendance à l’échelle que la revendication de simplicité cherchait à minimiser. Cette dépendance est d’autant plus problématique que les modèles propriétaires concernés évoluent régulièrement, rendant la reproduction exacte des résultats incertaine à moyen terme.

Par ailleurs, le nombre de clusters K est fixé à sa valeur réelle dans toutes les expériences sur les datasets d’intentions (77, 150, 89). Cette hypothèse est irréaliste en conditions de déploiement où K est inconnu et doit être estimé. Tout le défi de la sélection du nombre de clusters est ainsi externalisé. La méthode Log-Jump est utilisée pour la canonicalisation mais pas pour les autres tâches. Le comportement des méthodes guidées par LLM lorsque K est mal estimé n’est pas étudié, alors qu’il conditionne directement la pertinence pratique de l’approche.

3.3.2 Coût et scalabilité de l’argument économique

L’argument économique central de l’article mérite d’être situé dans son contexte temporel. Depuis la rédaction de l’article (mi-2023), les tarifs d’API ont diminué d’un facteur approximatif de dix et les modèles successeurs sont sensiblement plus performants. L’équation économique est donc entièrement à réviser. Si cette évolution pourrait sembler favorable à l’approche, elle profite en réalité à l’ensemble des méthodes qui mobilisent des LLMs, laissant l’avantage comparatif revendiqué par les auteurs sans démonstration actualisée.

La scalabilité différentielle des méthodes est également occultée. Le keyphrase clustering a un coût linéaire en fonction du nombre de documents, tandis que PCKMeans a un coût fixe d’environ 20,000 appels indépendamment de la taille du corpus. Il existe donc un point de croisement au-delà duquel la hiérarchie économique des méthodes s’inverse : la méthode la plus rentable à petite échelle devient la plus coûteuse à grande échelle. Ce seuil n’est jamais caractérisé par les auteurs, alors qu’il conditionne directement la pertinence de l’approche en contexte de production. À cela s’ajoutent des coûts opérationnels que l’article n’évoque pas : latence des appels API en batch, gestion des échecs et des retries, orchestration du pipeline, autant de frictions réelles qui pèsent sur le bilan économique effectif.

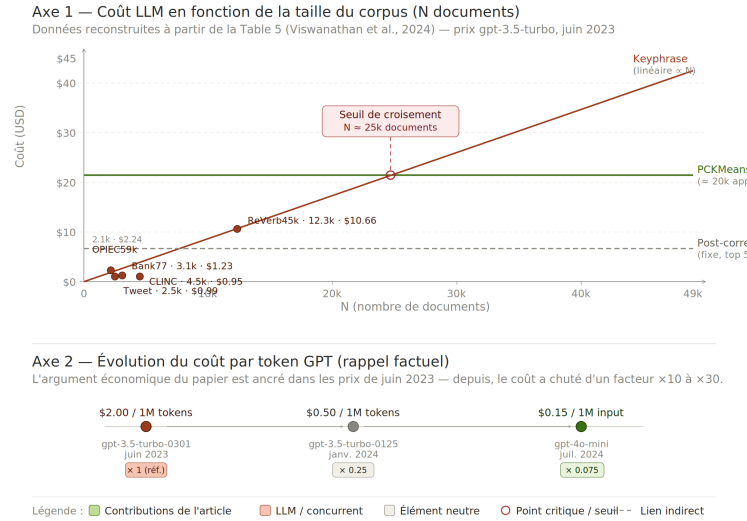


Fig. 3.3.1: Évolution du coût des méthodes LLM en fonction de la taille du corpus, mettant en évidence un seuil de rentabilité et la baisse significative du coût par token au fil du temps.

3.3.3 Adéquation des métriques au problème étudié

Les auteurs mobilisent deux familles de métriques : Macro F1, Micro F1 et Pairwise F1 pour la canonicalisation d'entités, accuracy et NMI pour le clustering d'intentions. Ces choix sont cohérents avec la littérature mais reposent tous sur une hypothèse forte : l'existence d'une unique partition de référence à laquelle comparer le partitionnement prédit.

Cette hypothèse entre en tension avec la motivation initiale de l'article, qui s'ouvre précisément sur le constat que le clustering est sous-spécifié et que plusieurs partitions peuvent légitimement satisfaire un même corpus. Évaluer la qualité d'un clustering par sa proximité à une partition unique revient à réintroduire implicitement la supervision que l'approche cherchait à minimiser. Sur Bank77 et CLINC, les catégories de référence proviennent de taxonomies d'intentions construites par les auteurs des datasets. Un utilisateur réel pourrait légitimement vouloir affiner ces taxonomies selon ses besoins et les métriques retenues pénaliseraient ces alternatives comme des erreurs. Pour un article qui revendique une amplification de l'intention utilisateur, l'absence d'évaluation avec des partitions multiples ou des jugements humains directs constitue une lacune notable.

3.4 Remarques sur les prompts

L'examen des prompts reproduits en annexes de l'article révèle deux observations complémentaires. D'une part, les démonstrations sont systématiquement construites sur des cas difficiles : abréviations, polysémie, institutions homonymes, ce qui relève davantage d'un prompt engineering soigné que d'un échantillonnage few-shot générique. La performance rapportée dépend donc de la qualité de cette curation, laquelle n'est ni systématisée ni testée en faisant varier le choix des exemples. Un chercheur utilisant quatre démonstrations différentes pourrait obtenir des résultats sensiblement différents, sans que l'article ne fournisse de cadre pour borner cet écart.

D'autre part, les prompts sont longs : de 500 à 700 tokens avec démonstrations, ce qui, à raison de 20,000 paires, engendre environ 12 millions de tokens en entrée pour la seule phase Pendant. Aucune optimisation de la longueur du prompt n'est évaluée, alors que l'ablation de la Table 3 suggère précisément que les démonstrations apportent peu de gain marginal par rapport à l'instruction seule. Ce constat invite à questionner la nécessité de prompts aussi élaborés et à envisager des variantes plus économiques sans perte significative de performance.

4 Évaluations complémentaires

Les sections précédentes ont identifié plusieurs limites méthodologiques de l'article de Viswanathan et al. : dépendance à des encodeurs datés, absence de mesure de la cohérence logique des contraintes LLM, absence de baseline non-LLM pour la post-correction et évaluation restreinte à des textes courts en anglais. Cette section confronte chacune de ces limites à une évaluation empirique dédiée.

Chaque évaluation cible un stade précis du pipeline proposé par les auteurs et teste une hypothèse causale sur le rôle du LLM :

Stade du pipeline	Hypothèse testée	Évaluation	Question posée
Avant (enrichissement par keyphrases)	Le LLM compense des représentations insuffisantes	4.1 : Ablation encodeurs	Le gain vient-il du LLM ou d'un encodeur daté ?
Pendant (PCKMeans)	Le LLM produit des contraintes fiables	4.2 : Cohérence logique des contraintes	Les contraintes sont-elles logiquement cohérentes et informativement riches ?
Après (post-correction)	Le LLM corrige les erreurs résiduelles	4.3 : Baseline géométrique	Une règle géométrique simple suffit-elle, sans LLM ?
Transversal	L'approche est générale	4.4 : Généralisation hors distribution	Les conclusions tiennent-elles sur un dataset différent ?

Note de reproductibilité. Le modèle original utilisé par les auteurs (gpt-3.5-turbo-0301) n'est plus accessible via l'API OpenAI. Afin de valider un substitut, nous avons conduit une comparaison pilote sur CLINC (30 puis 100 requêtes) entre gpt-3.5-turbo (version courante) et gpt-5.4-nano : les deux modèles produisent des performances quasi-identiques en accuracy et NMI, ce qui valide la substituabilité. Toutes les évaluations finales rapportées ci-dessous utilisent exclusivement gpt-5.4-nano, retenu comme alternative économique et documentée à gpt-3.5-turbo-0301. Les résultats ci-dessous sont obtenus sur CLINC, qui est le dataset pour lequel notre pipeline de reproduction est le plus complet.

4.1 Ablation encodeurs : le gain vient-il du LLM ou de la représentation ?

4.1.1 Motivation

L'article utilise Instructor (2022) pour Bank77/CLINC et DistilBERT-nli pour Tweet, des encodeurs qui ne représentent plus l'état de l'art en 2026. Si un encodeur moderne produit des embeddings suffisamment discriminants, le signal LLM pourrait devenir redondant. Cette évaluation vise à dissocier l'apport du LLM de l'apport de la représentation de base.

4.1.2 Protocole

Nous avons reproduit le pipeline PCKMeans pseudo-oracle sur CLINC en substituant l'encodeur original (Instructor) par trois alternatives :

- *distilbert-base-nli-stsb* : encodeur léger, utilisé dans l'article pour la tâche Tweet. Sert de borne inférieure.
- *all-MiniLM-L6-v2* : encodeur compact populaire, intermédiaire en performance.
- *mxbai-embed-large-v1* : encodeur récent (2024), parmi les meilleurs du benchmark MTEB au moment de nos expériences.

Pour chaque encodeur, nous comparons la baseline K-means (sans LLM) au clustering PCKMeans guidé par le pseudo-oracle LLM (500 contraintes).

4.1.3 Résultats

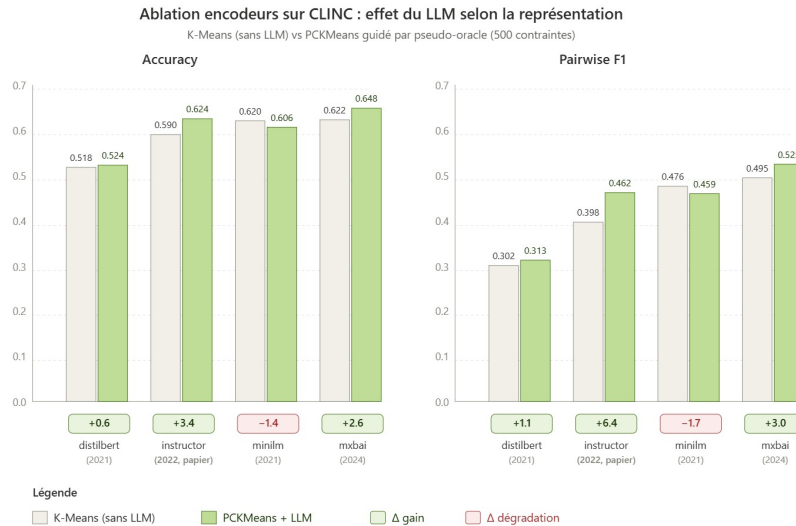


Fig. 4.1.1: Effet du pseudo-oracle LLM sur l'accuracy et le pairwise F1 selon l'encodeur (CLINC, 500 contraintes). K-means sur mxbai seul (0.622) atteint le niveau d'Instructor + LLM (0.624) sans aucun appel API.

4.1.4 Analyse

Trois observations émergent de ces résultats.

Premièrement, l'effet du LLM n'est pas universel : il dépend de l'encodeur de base. Le pseudo-oracle améliore nettement le clustering sur instructor et mxbai mais le dégrade sur minilm. Cela signifie que la conclusion principale de l'article « le LLM améliore le clustering » doit être conditionnée : elle est vraie pour certains encodeurs et fausse pour d'autres. Les auteurs ne testent qu'un seul encodeur par tâche et ne peuvent donc pas observer cette dépendance.

Deuxièmement, un encodeur moderne sans LLM atteint déjà le niveau de l'encodeur du papier avec LLM. C'est le résultat le plus structurant de cette ablation. K-means sur mxbai seul (accuracy 0.622) égale quasiment la performance d'Instructor avec pseudo-oracle LLM (accuracy 0.624). Dit autrement : le gain que l'article attribue au LLM peut être obtenu gratuitement en mettant à jour l'encodeur. Le LLM avec mxbai pousse encore le score à 0.648, soit un gain résiduel de +2.6 points, mais ce gain est modeste comparé au +3.2 points obtenus en changeant simplement d'encodeur sans aucun appel LLM. L'enrichissement par LLM compense donc principalement un encodeur daté plutôt qu'il n'ajoute un signal sémantique irremplaçable.

Troisièmement, la dégradation sur minilm révèle un mécanisme d'interférence. Lorsque l'encodeur produit déjà un espace bien structuré, des contraintes LLM bruitées peuvent « tordre » cet espace plutôt que l'améliorer. Ce phénomène est cohérent avec la sensibilité de PCKMeans au bruit de l'oracle, identifiée dans notre analyse critique (section 3.2.2).

Implication pour l'article. La valeur du LLM n'est pas absolue : elle est conditionnée par l'encodeur. L'article ne teste qu'Instructor, ce qui crée un biais favorable. Avec une baseline plus forte, l'écart se réduit. Et avec minilm, le LLM peut même dégrader la performance. Le gain est réel dans certaines configurations mais il ne constitue pas la contribution sémantique universelle que la présentation de l'article laisse supposer.

4.1.5 Saturation du budget de contraintes

Nous avons également testé le passage de 500 à 1,000 contraintes sur les deux meilleurs encodeurs. Le résultat est une *saturation* : le score de mxbai reste identique (acc = 0.648, pair. F1 = 0.525) et le F1 pairwise d'instructor se dégrade légèrement (0.462 → 0.449). Ce constat rejoint l'observation de la Figure 5 de l'article où la courbe de performance sature bien avant 20,000 contraintes. Il confirme que *le goulot d'étranglement n'est pas le volume de contraintes mais leur qualité et leur diversité informationnelle*.

4.2 Cohérence logique des contraintes : un signal monotone

4.2.1 Motivation

Le pseudo-oracle LLM génère des contraintes par paires (must-link ou cannot-link) qui alimentent PCKMeans. L'article ne mesure jamais la cohérence logique de ces contraintes. Or, si le LLM produit des jugements contradictoires, par exemple ML(A,B), ML(B,C) mais CL(A,C), l'algorithme reçoit un signal incohérent qui peut dégrader le clustering. Cette évaluation quantifie la fréquence des incohérences transitives et surtout caractérise la structure informationnelle du signal.

4.2.2 Protocole

Pour chaque run PCKMeans, nous avons enregistré l'intégralité des décisions du pseudo-oracle (must-link, cannot-link, ou abstention) dans un journal structuré. Nous construisons ensuite un graphe de contraintes où les nœuds sont les points du dataset et les arêtes portent le label de la décision LLM. Une *contradiction transitive* est un triangle (A, B, C) où ML(A,B) et ML(B,C) mais CL(A,C), c'est-à-dire une violation de la transitivité de la relation « même cluster ».

4.2.3 Résultats

Run	Budget	Must-link	Cannot-link	Abstentions	Contradictions	Taux
CLINC / instructor / 1 000	1000	2	997	1	0	0.000
CLINC / mxbai / 1 000	1000	1	997	2	0	0.000
CLINC / instructor / 500	500	1	497	4	0	0.000
CLINC / mxbai / 500	500	1	499	0	0	0.000
20 Newsgroups (OOD)	30	6	12	12	1	0.250

4.2.4 Analyse

Le résultat le plus frappant de cette évaluation n'est pas l'absence de contradictions mais la raison de cette absence. Sur CLINC, le pseudo-oracle produit quasi exclusivement des cannot-link : sur 1,000 contraintes, on observe typiquement 1 à 2 must-link, soit un ratio inférieur à 0,2 %. Un signal composé à 99,8 % de cannot-link est logiquement cohérent par construction (il est presque impossible de former un triangle incohérent quand presque toutes les arêtes sont de même signe), mais il est informativement pauvre.

Pourquoi cette distribution est-elle problématique ? Rappelons que CLINC contient 150 catégories d'intention. Avec autant de clusters, la probabilité qu'une paire sélectionnée aléatoirement soit un must-link est très faible ($\sim 1/150 \approx 0,7\%$). La stratégie Explore-Consolidate, qui sélectionne des paires diversifiées dans l'espace d'embedding, renforce encore ce déséquilibre : les paires diversifiées sont presque toujours des cannot-link. Le pseudo-oracle ne fait alors que confirmer ce que la géométrie de l'espace dit déjà : il n'ajoute presque aucune information positive (« ces deux points devraient être ensemble »).

En contraste, sur 20 Newsgroups (évaluation OOD, 30 contraintes seulement, 4 catégories proches), la distribution est plus équilibrée (6 ML, 12 CL, 12 abstentions) et une contradiction transitive apparaît immédiatement. Cela indique que l'audit de cohérence devient informatif dès que le signal n'est plus trivialement dominé par les cannot-link, c'est-à-dire dès que le LLM doit prendre des décisions non triviales.

Implication pour l'article. Les auteurs rapportent une précision globale du pseudo-oracle de 87 à 97 % selon les datasets. Notre analyse montre que cette précision élevée peut être trompeuse : sur CLINC, elle est largement portée par des cannot-link faciles que n'importe quelle heuristique géométrique pourrait aussi deviner. La véritable question n'est pas « le LLM a-t-il raison ? » mais « le LLM apporte-t-il de l'information que l'encodeur n'avait pas déjà ? ». Sur CLINC à 150 clusters, la réponse semble être : très peu.

4.3 Baseline géométrique : le LLM est-il nécessaire pour la post-correction ?

4.3.1 Motivation

L'article propose une post-correction où un LLM réexamine les points de faible marge et les réassigne éventuellement à un cluster voisin. Mais cette méthode n'est jamais comparée à une baseline non-LLM. Si une règle géométrique simple donne des résultats comparables, l'apport spécifique du LLM en post-correction est négligeable et les gains rapportés ne sont que l'effet de la reconsidération des cas incertains, pas de la compréhension sémantique du modèle.

4.3.2 Protocole

Notre baseline géométrique procède comme suit, sans aucun appel LLM :

1. Identification des points incertains. Comme dans l'article, nous sélectionnons les k points présentant la plus petite marge, c'est-à-dire la plus petite différence de distance entre leur premier et leur deuxième cluster le plus proche. Nous utilisons $k = 100$ en mode pilote.
2. Représentation de chaque cluster candidat. Pour chaque point incertain, nous considérons les 5 clusters les plus proches. Chacun de ces clusters est représenté par ses 3 points de plus faible inertie (les 3 points les plus proches du centroïde) qui servent de prototypes du cluster.
3. Décision de réassignation. Nous calculons la similarité moyenne entre le point incertain et les 3 représentants de chaque cluster candidat. Si le gain de similarité par rapport au cluster actuellement assigné dépasse un seuil fixé à 0.02, le point est réassigné au cluster candidat offrant la plus forte similarité moyenne.

Ce protocole est directement comparable à la correction LLM de l'article, qui suit la même logique (identification par marge, représentation par prototypes, interrogation séquentielle des clusters candidats) mais remplace le calcul de similarité par un appel LLM.

4.3.3 Résultats

Deux configurations ont été testées sur CLINC, chacune avec un encodeur différent :

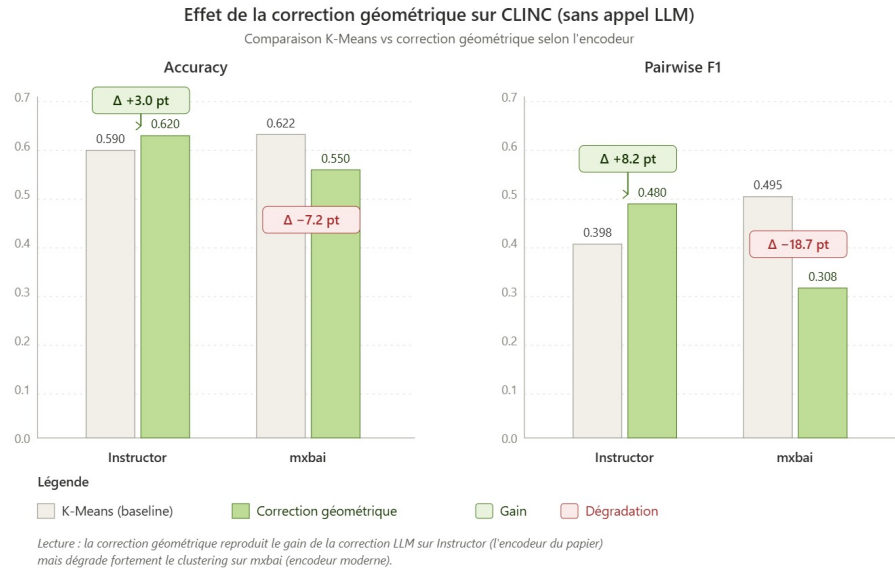


Fig. 4.3.1: Effet de la correction géométrique sur CLINC selon l'encodeur. Sur Instructor (encodeur du papier), la baseline non-LLM produit un gain comparable à la correction LLM rapportée par l'article. Sur mxbai (encodeur moderne), la même règle dégrade fortement le clustering.

4.3.4 Analyse

Les résultats montrent un contraste marqué selon l'encodeur.

Avec Instructor, la correction géométrique produit un gain substantiel (+3.0 points d'accuracy, +8.2 points de pairwise F1). Dans ce régime, le clustering initial est de qualité moyenne (accuracy 0.59) et beaucoup de points sont effectivement mal assignés. Leurs deuxièmes clusters les plus proches sont souvent les bons et la règle géométrique (qui ne coûte aucun appel LLM) suffit à corriger une partie significative des erreurs. Ce gain est comparable à celui de la correction LLM rapportée par l'article (+0.1 à +5.2 points selon les datasets). La correction géométrique reproduit donc l'essentiel du bénéfice de la correction LLM sur l'encodeur du papier, ce qui signifie que sur cette configuration, le gain attribué au LLM provient principalement du fait de reconsidérer les points incertains, pas de la compréhension sémantique du modèle.

Avec mxbai, la correction géométrique dégrade fortement le clustering (-7.2 points d'accuracy, -18.7 points de pairwise F1). Ce résultat, à première vue surprenant, s'explique par la structure différente de l'espace d'embedding. Dans l'espace mxbai, les distances entre clusters ne reflètent pas les mêmes proximités sémantiques que dans l'espace Instructor. La règle géométrique, qui réassigne aveuglément au cluster le plus similaire en distance, fait des réassignations qui cassent la structure plutôt que de l'améliorer. Un LLM pourrait en principe éviter ces erreurs grâce à sa compréhension du contenu,

mais les données de la Table 4 de l’article montrent que le LLM n’atteint lui-même que 55–57 % de précision sur les réassignations (OPIEC59k et CLINC), ce qui limite cet avantage théorique.

Implication pour l’article. La correction LLM est la méthode la plus faible du triptyque proposé par Viswanathan et al., avec des gains modestes de +0.1 à +5.2 points. Notre baseline géométrique montre que sur l’encodeur du papier, ces gains sont reproductibles sans aucun appel LLM. Toutefois, la dégradation observée avec mxbai montre aussi que la correction géométrique aveugle comporte des risques propres. La conclusion n’est donc pas que la post-correction LLM est inutile, mais qu’elle n’a pas été comparée aux alternatives les plus évidentes, ce qui fragilise l’attribution causale de ses gains à la compréhension sémantique du modèle.

4.4 Généralisation hors distribution : 20 Newsgroups

4.4.1 Motivation

Les cinq datasets du papier partagent plusieurs caractéristiques restrictives : textes courts (5–20 mots), anglais, orientés dialogue ou entités. Pour évaluer la robustesse des conclusions, nous testons l’approche sur un dataset structurellement différent : *20 Newsgroups*, un classique du clustering de texte composé de posts de forums de discussion (textes longs, vocabulaire varié, 20 catégories thématiques).

Nous utilisons un sous-ensemble de 20 Newsgroups restreint à des catégories proches (que nous notons 20ng_diff3) pour évaluer le régime le plus difficile, celui où les clusters sont sémantiquement voisins.

4.4.2 Résultats préliminaires

Algorithme	Accuracy	NMI	Pairwise F1	Taux d’abstention
K-Means	0.752	0.478	0.644	—
PCKMeans pseudo-oracle LLM	0.816	0.489	0.687	0.40

4.4.3 Analyse

Le pseudo-oracle LLM améliore l’accuracy de +6.3 points et le pairwise F1 de +4.3 points par rapport à la baseline K-means. C’est un signal encourageant : l’approche peut transférer au-delà de CLINC et des datasets courts de l’article.

Cependant, deux signaux de fragilité tempèrent cet optimisme. D’une part, le taux d’abstention atteint 40 %, le LLM ne parvient pas à trancher pour près de la moitié des paires présentées. Sur CLINC, ce taux ne dépasse pas 0,1 à 0,2 %. Cette différence s’explique par la longueur et la complexité des textes de 20 Newsgroups : des posts de forum de plusieurs paragraphes sont plus difficiles à comparer que des requêtes de 10 mots. D’autre part, l’audit de cohérence (section 4.2) détecte une contradiction transitive sur seulement 30 contraintes, soit un taux de 25 % des triangles formés, ce qui est bien supérieur aux 0 % observés sur CLINC.

Ces résultats préliminaires doivent être interprétés avec prudence en raison de leur échelle limitée (30 contraintes). Ils indiquent néanmoins que la généralisation est possible mais plus fragile : le LLM aide davantage (gain plus élevé) tout en étant moins fiable (plus d’abstentions et d’incohérences). Une évaluation à plus grande échelle serait nécessaire pour confirmer cette tendance.

4.5 Synthèse des évaluations complémentaires

Les quatre évaluations convergent vers une lecture cohérente et plus nuancée que les conclusions de l’article.

Le gain du LLM compense principalement un encodeur daté. L’ablation encodeurs (§4.1) révèle qu’un encodeur moderne (mxbai, 2024) sans aucun appel LLM atteint déjà le niveau de performance de l’encodeur du papier (Instructor, 2022) assisté par le pseudo-oracle. Le LLM ajoute encore un gain résiduel avec mxbai (+2.6 points) mais ce gain est modeste comparé au +3.2 points obtenus en changeant simplement d’encodeur. Sur minilm, le LLM dégrade même la performance. L’article ne teste qu’un encodeur par tâche et ne peut donc pas observer cette dépendance, ce qui produit un biais favorable à ses conclusions.

Le volume de contraintes n’est pas le facteur limitant : c’est leur qualité. Le passage de 500 à 1,000 contraintes ne produit aucun gain mesurable (§4.1) et l’audit de cohérence (§4.2) révèle que sur CLINC, le pseudo-oracle produit quasi-exclusivement des cannot-link (99,8 %). Ce signal logiquement cohérent mais informativement pauvre ne fait que confirmer ce que la géométrie de l’espace d’embedding indique déjà. Le facteur limitant n’est donc pas le nombre de requêtes LLM mais la diversité et l’équilibre des contraintes générées.

La post-correction par LLM n’est pas clairement supérieure à une règle géométrique. Sur l’encodeur du papier, la baseline non-LLM (§4.3) produit un gain comparable à celui de la correction LLM (+3.0 points sans aucun appel LLM), ce qui

suggère que le bénéfice rapporté par l'article provient du simple fait de reconsidérer les points incertains, pas de la compréhension sémantique du modèle. Sur mxbai en revanche, la correction géométrique dégrade fortement le clustering (-7.2 points), ce qui montre que la stratégie n'est pas universellement sûre.

La généralisation hors distribution est possible mais fragile. Sur 20 Newsgroups (§4.4), le pseudo-oracle LLM produit un gain d'accuracy de $+6.3$ points, ce qui est encourageant. Mais le taux d'abstention de 40 % et l'apparition de contradictions transitives signalent une fiabilité moindre du LLM sur des textes plus longs et plus complexes que ceux des datasets de l'article.

En résumé, nos évaluations ne remettent pas en cause l'intuition centrale de Viswanathan et al. : un LLM peut effectivement améliorer le clustering, mais elles montrent que cette amélioration est largement substituable par une mise à jour de l'encodeur, saturante avec le nombre de contraintes, portée par un signal monotone dominé par les cannot-link faciles et fragile hors du régime expérimental de l'article.

Conclusion

L'article défend une idée forte : un LLM peut servir de source de supervision ciblée dans un pipeline de clustering sans qu'il soit nécessaire de reconstruire le pipeline autour d'un modèle profond spécialisé. Le cadre before / during / after qu'ils proposent fournit une grille d'analyse claire et réutilisable et le résultat selon lequel l'enrichissement par keyphrases (stade before) est la méthode la plus robuste et la plus économique constitue une contribution pratique solide.

Notre travail conduit toutefois à reformuler les conclusions de l'article de manière plus précise.

Sur la reproductibilité. Le modèle original gpt-3.5-turbo-0301 n'est plus accessible, ce qui rend la réplique exacte impossible. Nos expériences avec des modèles substitués montrent des performances proches mais non identiques, ce qui soulève une question plus large sur la pérennité des travaux construits sur des APIs commerciales fermées.

Sur la robustesse des gains. Nos évaluations complémentaires montrent que les gains rapportés par l'article sont largement substituables par une simple mise à jour de l'encodeur. Sur CLINC, un encodeur moderne (mxbai, 2024) sans aucun appel LLM atteint déjà le niveau de performance de l'encodeur du papier (Instructor, 2022) assisté par le pseudo-oracle. Le gain résiduel du LLM par-dessus un encodeur moderne existe ($+2.6$ points) mais reste modeste et il disparaît ou s'inverse selon l'encodeur choisi (dégradation de -1.4 points sur minilm). De plus, le signal produit par le pseudo-oracle sur CLINC est informativement pauvre : composé à 99,8 % de cannot-link, il ne fait que confirmer ce que la géométrie de l'espace d'embedding indique déjà.

Sur la post-correction. La correction LLM, troisième composante du framework, n'est pas clairement supérieure à une baseline géométrique qui ne coûte aucun appel LLM. Sur l'encodeur du papier, la correction géométrique reproduit l'essentiel du bénéfice rapporté par l'article, ce qui fragilise l'attribution causale des gains à la compréhension sémantique du modèle.

Sur la généralisation. L'extension hors distribution sur 20 Newsgroups suggère que l'approche peut transférer au-delà des datasets de l'article mais avec une fiabilité moindre : taux d'abstention élevé (40 %), apparition de contradictions transitives et signal plus bruité. Les cinq datasets de l'article, tous constitués de textes courts en anglais, ne capturent pas cette fragilité.

Sur le positionnement dans la littérature. Depuis la publication de l'article, plusieurs travaux (LUMI, Dial-In LLM, Huang & He) ont identifié des limitations structurelles de l'approche pseudo-oracle pairwise : notamment le bruit des pseudo-labels single-shot et la restriction aux décisions binaires. Notre analyse empirique converge avec ces critiques : la cohérence apparente des contraintes peut masquer un signal pauvre et l'attribution causale des gains au LLM nécessite des baselines non-LLM que l'article ne fournit pas.

Au final, l'apport principal de notre projet n'est pas seulement de reproduire les expériences du papier mais de les replacer dans un cadre expérimental plus transparent. Nos évaluations complémentaires montrent que l'approche de Viswanathan et al. fonctionne mais que sa valeur ajoutée est plus étroite que ne le suggère la présentation de l'article : elle compense principalement un encodeur daté, sature rapidement avec le nombre de contraintes et repose sur un signal dont la richesse informationnelle mérite d'être questionnée. C'est précisément ce que la littérature postérieure commence à établir et nous espérons que notre travail contribue modestement à cette clarification.